

2. Connexion JDBC de base

Étape 2 — EILMS : Database Persistence Layer (JDBC, PostgreSQL, JPA/Hibernate)

Fichier concerné : `src/main/java/com/eilms/config/DatabaseConfig.java`

Avant d'utiliser un framework ORM, il faut comprendre la connectivité brute à la base de données. Cette classe fournit une méthode simple pour ouvrir une connexion JDBC vers PostgreSQL.

```
public class DatabaseConfig {  
  
    private static final String URL = "jdbc:postgresql://localhost:5432/eilms_db";  
    private static final String USER = "postgres";  
    private static final String PASSWORD = "postgres_password";  
  
    public static Connection getConnection() throws SQLException {  
        return DriverManager.getConnection(URL, USER, PASSWORD);  
    }  
}
```

Explication détaillée

URL de connexion : `jdbc:postgresql://localhost:5432/eilms_db` suit le format standard JDBC : protocole + hôte + port (5432 = port par défaut de PostgreSQL) + nom de la base (« `eilms_db` »).

USER / PASSWORD : identifiants de connexion à la base. Ici en dur dans le code (acceptable en phase d'apprentissage), mais en production on les externaliserait dans des variables d'environnement ou un fichier de config sécurisé.

DriverManager.getConnection() : méthode statique du JDK qui négocie une session avec la base à partir de l'URL/USER/PASSWORD, et renvoie un objet `Connection` représentant la session ouverte.

Exemple d'utilisation : `try (Connection conn = DatabaseConfig.getConnection()) { ... }` — permet de tester que le serveur PostgreSQL répond bien, avant d'ajouter la couche ORM par-dessus.

Analyse pédagogique du professeur

Vérification de base : fournit un mécanisme léger pour tester la connectivité réseau directe au serveur PostgreSQL, sans la surcharge d'un framework.

Rôle de DriverManager : négocie les sessions de base de données. En production, cette classe est généralement enveloppée par un pool de connexions géré (`DataSource`), comme HikariCP.